SKILLBIT: COMPLETE OFFLINE PROJECT REVIEW, BACKEND CODING, DEBUGGING & VIVA HANDBOOK

Explainable AI Resume Builder, Deterministic ATS Job Matcher, Multi-Source AI Job Discovery Engine & Dual-OTP SMTP Authentication Platform

Purpose of this Handbook: This document is tailored specifically for your offline college project review (10:00 AM session). It covers: (1) System Architecture & End-to-End Code Flow, (2) Mathematical Formulas & AI/NLP Algorithms, (3) Real Debugging Scenarios we solved in this project and how to explain/fix them live, (4) Hands-on Backend Coding Recipes if the mentor asks you to write or modify code on your laptop, and (5) 30 Viva & Technical Interview Questions with crisp, high-scoring answers.

1. 60-Second Project Pitch & Pre-Review Laptop Checklist

How to introduce SkillBit in 60 seconds to the panel:
"SkillBit is a full-stack Career Intelligence Platform built with FastAPI, SQLAlchemy 2.0, Scikit-Learn, and React 19. Most resume tools fall into two flawed extremes: either shallow keyword counters that miss aliases like 'k8s' for 'Kubernetes' and ignore whether a skill is proven in work experience, or pure LLM wrappers that hallucinate fake metrics and give different scores on every run. SkillBit solves this with a **4-Layer Hybrid AI Architecture**: (1) A deterministic canonical taxonomy of 106+ skills and 370+ aliases with boundary-safe regex, (2) A 4-tier Evidence Tracer and hybrid Word TF-IDF + Token Jaccard + Character N-Gram semantic matcher, (3) A proactive Multi-Source AI Job Discovery Engine that aggregates the user's Master Profile and all saved resumes, flags cross-resume contradictions, and searches public APIs, company career boards, and community feeds, and (4) A 5-tier Multi-Provider LLM router (Groq, Gemini, Grok, OpenAI, plus a deterministic Local CareerTech Engine) so the system works 100% offline even without internet."

1.1 How to Start the Project on Your Laptop Before the Review

```
# Terminal 1: Start FastAPI Backend (on Port 8000, or 8001 if 8000 is busy)
cd C:\Users\user\Desktop\Skillbit\backend
..\venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8001

# Terminal 2: Start React 19 + Vite Frontend (pointing to Backend Port 8001)
cd C:\Users\user\Desktop\Skillbit\skillbit
$env:PORT="5174"; $env:VITE_API_URL="http://localhost:8001/api"; npm run dev

# Terminal 3: Run Automated Test Suite (46 Unit & Integration Tests)
cd C:\Users\user\Desktop\Skillbit\backend
..\venv\Scripts\python.exe -m pytest -v
```

Service / Check	Local URL	What to Show the Mentor
React 19 Web App	http://localhost:5174	Full UI: Resumes, Job Finder, Job Match (URL/PDF/TXT JD import), Roadmaps, Builder, Profile
Swagger OpenAPI Docs	http://localhost:8001/docs	All 50+ REST API endpoints across 10 modular routers; test live APIs
Health Check	http://localhost:8001/health	Returns {"status": "ok"}
SMTP Status Check	http://localhost:8001/api/auth/smtp-status	Shows Gmail SMTP connection status or automatic terminal mock fallback
Active LLM Provider	http://localhost:8001/api/match/llm-provider	Shows active LLM (Groq Llama/GPT-OSS) or Local CareerTech Engine

2. Complete Backend & Frontend Codebase Map

During an offline code review, mentors frequently ask: *"Show me the exact file where X is implemented."* Use this table to navigate directly to any file in seconds:

Feature / Layer	Exact File Path	Key Functions & Classes Inside
FastAPI Entrypoint & CORS	backend/app/main.py	lifespan(), CORSMiddleware, includes all 10 API routers under /api
Environment & Settings	backend/app/config.py	Settings(BaseSettings) loading backend/.env, JWT, DB, SMTP, LLM & Search keys
DB Session & Auto-Seeder	backend/app/database/session.py backend/app/database/init_db.py	engine, SessionLocal, get_db(), init_db(), schema migrations & skill seeder
JWT & Bcrypt Security	backend/app/utils/security.py backend/app/api/deps.py	get_password_hash(), verify_password(), create_access_token(), get_current_active_user()
Dual SMTP & OTP Auth	backend/app/api/auth.py backend/app/services/email_service.py	register(), verify_otp(), verify_email(), resend_verification(), send_verification_email()
PDF/DOCX/TXT Parser	backend/app/ai/resume_parser.py backend/app/ai/section_classifier.py	extract_text_from_pdf() (PyMuPDF block sort), classify_resume_sections()
Skill Normalizer & Tracer	backend/app/ai/skill_normalizer.py backend/app/ai/skill_extractor.py	normalize_single_skill(), extract_resume_skill_profile(), 4 evidence tiers
Hybrid TF-IDF Embeddings	backend/app/ai/embedding_service.py	SemanticEmbeddingService: Word TF-IDF (0.40) + Jaccard (0.35) + Char N-Gram (0.25)
ATS Fit & Gap Engine	backend/app/ai/matching_engine.py backend/app/ai/gap_analyzer.py	calculate_match_metrics(), analyze_skill_gaps() (STRONG, WEAK, PARTIAL, MISSING)
JD URL & File Extractor	backend/app/api/job.py	extract_job_from_url() (JSON-LD + HTML + PDF stream), extract_job_from_file()
Multi-Resume Intelligence	backend/app/ai/candidate_intelligence.py	build_candidate_profile_snapshot(), deduplication, provenance & conflict detection
Multi-Source Job Search	backend/app/services/job_sources.py backend/app/ai/job_discovery_agent.py	Remotive, Arbeitnow, Greenhouse, Hacker News, Reddit adapters + SSRF/XSS guards
7-Factor Job Ranker	backend/app/ai/job_matcher_ranker.py	evaluate_job_for_candidate(), mandatory eligibility hard cap, best resume selector
Multi-LLM Router	backend/app/ai/llm_service.py	LLMService: Groq -> Gemini -> Grok -> OpenAI -> Local CareerTech Fallback
ReportLab ATS PDF Engine	backend/app/services/pdf_service.py	build_ats_resume_pdf() single-column ATS PDF with clickable hyperlinks
Frontend State & API	skillbit/src/lib/api.ts skillbit/src/lib/store.tsx	Typed fetch wrapper with /api proxy fallback, JWT interceptor, session state
Frontend Pages	skillbit/src/pages/JobFinder.tsx skillbit/src/pages/NewMatch.tsx skillbit/src/pages/Builder.tsx	AI Job Finder UI, Job Match with URL/PDF/TXT JD importer, A4 Resume Builder

3. Mathematical Formulas & AI/NLP Pipeline (Viva Deep Dive)

3.1 Boundary-Safe Canonical Skill Normalization

Problem: Naive substring matching matches 'Java' inside 'JavaScript' or 'C' inside 'CSS', and fails to match 'C++' or '.NET' because Python's standard \b word boundary treats '+' and '.' as non-word characters.
Our Solution (skill_extractor.py): We escape special characters via re.escape(term) and use custom negative lookbehind and lookahead assertions:

```
escaped = re.escape(term)  # e.g., 'C++' -> 'C\+\+'
pattern = re.compile(rf"(?<![a-zA-Z0-9+#]){escaped}(?![a-zA-Z0-9+#])", re.IGNORECASE)
```

3.2 Four-Tier Evidence Verification Multipliers

Instead of treating a skill listed in a footer the same as a skill proven in work experience, skill_extractor.py inspects the classified resume section and checks for 19+ high-impact action verbs (*architected, engineered, built, deployed, optimized, scaled*):

Evidence Tier	Multiplier	Detection Condition in Resume
STRONG	1.00x (100%)	Found in Experience or Projects section AND sentence contains an Action Verb
MODERATE	0.85x (85%)	Found in Experience or Projects section without a strong action verb
LISTED	0.60x (60%)	Listed only in the raw Technical Skills section (no project/work proof)
MENTIONED	0.40x (40%)	Mentioned casually in Summary or Header only

3.3 Hybrid Subword Semantic Similarity (embedding_service.py)

To measure semantic similarity between a job requirement snippet A and a candidate bullet point B without requiring a heavy 2GB GPU transformer model, we combine three complementary similarity metrics:

```
S_semantic(A, B) = 0.40 * Cosine_Word_TFIDF(1..2 ngrams)
                 + 0.35 * Jaccard_Token_Overlap(|Tokens_A n Tokens_B| / |Tokens_A u Tokens_B|)
                 + 0.25 * Cosine_Char_WB_TFIDF(3..5 subword character n-grams)
```

Why Character N-Grams (analyzer='char_wb', ngram_range=(3,5))? Because character n-grams capture morphological stems and subwords (e.g., matching *'containerized'* with *'containers'*, or *'microservices'* with *'services'*) even when exact words differ!

3.4 Deterministic ATS Job Fit Score Formula (matching_engine.py)

```
For each Job Skill i:
  Weight W_i = 1.0 if REQUIRED section, else 0.5 if PREFERRED section.
  If skill i is in Candidate Canonical Skills:
      Alignment F(i) = 0.60 * 1.0 + 0.40 * max(Evidence_Multiplier, S_semantic)
  Else if S_semantic >= 0.70 (Transferable experience):
      Alignment F(i) = 0.45 * S_semantic
  Else:
      Alignment F(i) = 0.0

Overall Fit Score = min(100.0, (Sum(W_i * F(i)) / Sum(W_i)) * 100)
```

3.5 AI Job Discovery 7-Factor Ranking & Mandatory Eligibility Hard Cap

Dimension	Weight	How It Is Calculated
1. Skill Compatibility	25%	Required (1.0) & Preferred (0.5) canonical skill match + evidence tier + semantic match
2. Experience & Projects	20%	Project technology overlap (55%) + experience bullet TF-IDF similarity (45%)
3. Role Alignment	15%	Similarity between job title and user's preferred/inferred/adjacent roles
4. Education & Eligibility	15%	Degree compatibility + candidate years of experience vs job minimum years

Dimension	Weight	How It Is Calculated
5. Location & Work Mode	10%	Remote / Hybrid / On-site alignment with user preferences
6. Freshness & Availability	10%	Recently Published (100), Active (92), Closing Soon (85), Possibly Expired (35)
7. Other User Preferences	5%	Internship vs Full-time preference, salary floor, and excluded companies/categories

Mandatory Eligibility Hard Cap Rule: If a job has an unmet mandatory requirement (e.g., requires 8+ years Staff experience while the candidate has 0 years, or is strictly On-site when the user enabled remote_only), the status is set to Not eligible based on available information and the score is hard-capped at **<= 42.0%** so it never outranks eligible roles.

4. Real Error Debugging Playbook (Bugs We Solved & Mentor Scenarios)

The college announcement specifically states: *"The upcoming project review will include backend coding, error debugging, and hands-on problem-solving."* Below are the exact bugs we diagnosed and fixed in this codebase, plus common errors mentors test:

Bug #1: SMTP Email Not Sending / Nested Angle Brackets in From Header

Root Cause: In backend/.env, SMTP_FROM_EMAIL was set to "PixelHub <shishirplays@gmail.com>". In email_service.py, the code wrapped it a second time: f"{from_name} <{from_email}>", creating an invalid header "SkillBit <PixelHub <shishirplays@gmail.com>>". Furthermore, uvicorn cached the initial Settings() object from before .env was edited.

How We Fixed It: In backend/app/services/email_service.py (get_clean_smtp_credentials), we now call Settings() dynamically on each dispatch, strip quotes/spaces from Google App Passwords, and use email.utils.parseaddr() and formataddr() so both 'email@domain.com' and 'Name <email@domain.com>' work cleanly.

Bug #2: Frontend OTP Verification Crash (TypeError: api.setToken is not a function)

Root Cause: In skillbit/src/lib/store.tsx, verifyOtp() and verifyEmail() called api.setToken(res.access_token). However, setToken was a standalone exported function in api.ts, not a property on the api object, causing a runtime TypeError on OTP submit.

How We Fixed It: We attached getToken and setToken to the exported api object in skillbit/src/lib/api.ts AND updated verifyOtp() and verifyEmail() in store.tsx to call setToken(res.access_token) followed by await startSession(backendUser) so user workspace data hydrates immediately.

Bug #3: Port 8000 / 5173 Conflict ([WinError 10048] Address already in use)

Root Cause: Another process (e.g., python -m http.server 8000 or another Vite app on 5173) was already bound to the default port. If the frontend still calls http://localhost:8000/api, it hits the wrong server!

How We Fixed It: 1) Check listening ports: Get-NetTCPConnection -State Listen 2) Launch backend on an available port (--port 8001). 3) Launch frontend with $env:PORT='5174'; $env:VITE_API_URL='http://localhost:8001/api'; npm run dev. 4) FastAPI's CORSMiddleware uses allow_origin_regex=r'^https?://(localhost|127\.0\.0\.1)(:\d+)?$' so any localhost port is allowed automatically.

Bug #4: 1-Click Email Verification Link Pointing to Wrong Port (5173 vs 5174)

Root Cause: When the frontend ran on port 5174, /api/auth/register used hardcoded settings.FRONTEND_URL (http://localhost:5173), sending users to the wrong port when clicking the email link.

How We Fixed It: In backend/app/api/auth.py, register() and resend_verification() now inspect request.headers.get('origin') first, dynamically building the verification link for the exact frontend port.

Bug #5: SQLite 'OperationalError: no such column' After Pulling New Branch

Root Cause: SQLAlchemy's Base.metadata.create_all(bind=engine) creates missing tables, but does NOT alter existing tables when new columns (like users.is_verified or users.verification_otp) are added.

How We Fixed It: In backend/app/database/init_db.py, we use SQLAlchemy's inspect(engine).get_columns('users') on startup and execute ALTER TABLE users ADD COLUMN ... automatically for any missing column.

Bug #6: SSRF & Prompt Injection When Importing External Job URLs

Root Cause: If a user pastes http://127.0.0.1:8001/admin or http://169.254.169.254 into the Job URL importer, a naive backend httpx.get(url) would expose internal services (SSRF). Or a malicious webpage could include 'Ignore previous instructions'.

How We Fixed It: validate_external_url() in backend/app/services/job_sources.py parses the URL, blocks non-http/https schemes, blocks localhost/loopback/private IPs via Python's ipaddress module, and sanitize_untrusted_job_content() strips scripts and prompt-injection directives.

5. Hands-On Backend Coding Cheat Sheet (Live Coding Tasks)

During the offline review, mentors often ask you to make a small live code change to prove you wrote and understand the codebase. Here are the 4 most common live coding tasks and the exact code to write:

Task A: "Add a new API endpoint in FastAPI (e.g., GET /api/skills/count)"

Open backend/app/api/skills.py and add:

```
@router.get("/count")
def get_total_skills_count(db: Session = Depends(get_db)):
    total_skills = db.query(Skill).count()
    return {"total_canonical_skills": total_skills}
```

Task B: "Add a new Canonical Skill and Aliases to the Taxonomy"

You can either add it via Swagger UI (POST /api/admin/skills) OR in backend/app/utils/taxonomy_data.py inside CANONICAL_SKILLS:

```
{
    "name": "LangChain",
    "category": "AI / Machine Learning",
    "aliases": ["langchain", "lang-chain", "langchain.py"],
    "description": "Framework for developing applications powered by language models."
}
```

Task C: "Change the Required vs Preferred Skill Weights in Job Matching"

Open backend/app/ai/job_analyzer.py (lines 121-124): Required skills are scanned with weight=1.0 and Preferred skills with weight=0.5. And in backend/app/ai/matching_engine.py (line 64), exact match is weighted 0.6 and evidence/semantic is weighted 0.4:

```
# backend/app/ai/matching_engine.py (Line 64)
combined_factor = 0.6 * 1.0 + 0.4 * max(evidence_factor, semantic_sim)
```

Task D: "Show how the JD Link & PDF/TXT Importer works in Job Match"

Open backend/app/api/job.py and show extract_job_from_file (Line 65) and extract_job_from_url (Line 97):

```
# 1. File Upload (/api/job/extract-file):
#    Saves temp file -> extract_text_from_file() (PyMuPDF/docx/txt) ->
#    _infer_title_and_company_from_jd_text() -> analyze_job_requirements()

# 2. URL Import (/api/job/extract-url):
#    validate_external_url() (SSRF check) -> httpx.Client.get() ->
#    Parses application/ld+json JobPosting schema or HTML/PDF stream ->
#    Auto-populates Role Title, Company, and JD Text in NewMatch.tsx!
```

6. Comprehensive Viva & Technical Interview Q&A Bank (25 Questions)

Q1. Why did you choose a Hybrid AI architecture instead of using a single ChatGPT/LLM prompt to score resumes?

Pure LLM prompts are non-deterministic (they give different scores for the same resume on repeated runs), prone to hallucinating skills or fake metrics, slow, and fail when API rate limits hit. SkillBit uses deterministic NLP and canonical taxonomy matching for 100% reproducible scoring, and uses LLMs strictly for executive summaries, STAR bullet phrasing, and learning roadmaps with a deterministic local fallback.

Q2. How does SkillBit extract text from multi-column or complex PDFs without scrambling lines?

In backend/app/ai/resume_parser.py, we use PyMuPDF (fitz) with page.get_text('blocks', sort=True). This sorts text blocks by their (y0, x0) bounding-box coordinates top-to-bottom and left-to-right, unwraps hyphenated line breaks (e.g., 'imple-\nmentation' -> 'implementation'), and normalizes Unicode bullet symbols.

Q3. How does your Section Classifier know which part of a resume is Experience vs Projects vs Skills?

In backend/app/ai/section_classifier.py, we scan lines for structural header patterns (e.g., 'Work Experience', 'Professional Experience', 'Technical Projects', 'Education', 'Skills') using compiled regular expressions on short header lines (< 45 chars), partitioning the document into typed text buffers before skill extraction.

Q4. Why do you grade skills into STRONG, MODERATE, LISTED, and MENTIONED evidence tiers?

Real ATS parsers and engineering managers value demonstrated impact over keyword stuffing. If a candidate only writes 'Docker' in their Skills list, it gets LISTED (0.60x). If they write 'Deployed microservices using Docker, cutting release time by 50%' in Experience, our extractor detects the section ('experience') and action verb ('deployed') and awards STRONG (1.00x) evidence.

Q5. Explain how your Semantic Embedding Service works without requiring a GPU.

In backend/app/ai/embedding_service.py, SemanticEmbeddingService combines three Scikit-Learn/NLP signals: 40% Word N-Gram TF-IDF cosine similarity (1-2 word phrases), 35% stop-word filtered Token Jaccard overlap, and 25% Character Word-Boundary N-Gram TF-IDF (3-5 char subwords). It runs in <15ms on CPU with zero external API latency.

Q6. What are the four Skill Gap states and how are priorities assigned?

In gap_analyzer.py: (1) STRONG_MATCH (Priority: LOW) when a skill has STRONG/MODERATE evidence; (2) WEAK_EVIDENCE (Priority: HIGH if Required, MEDIUM if Preferred) when a skill is only LISTED/MENTIONED without work bullets; (3) PARTIAL_MATCH (Priority: MEDIUM) when exact skill is missing but semantic similarity >= 0.70; (4) MISSING (Priority: HIGH if Required, MEDIUM if Preferred) when absent and semantic similarity < 0.70.

Q7. How does the Multi-Provider LLM Failover Router work in llm_service.py?

LLMService checks configured API keys in priority order: (1) Groq Cloud (openai/gpt-oss-120b, llama-3.3-70b-versatile), (2) Google Gemini 1.5 Flash, (3) xAI Grok, (4) OpenAI GPT-4o-mini, and (5) SkillBit Local CareerTech Engine. If any cloud provider times out or returns invalid JSON, it automatically cascades to the next provider without crashing the request.

Q8. How do you prevent LLM hallucination when rewriting resume bullets?

In resume_content_generator.py, our STAR optimizer upgrades passive verbs (e.g., 'worked on' -> 'engineered') and inserts explicit bracketed placeholders like [X]% or [metric] when numbers are absent, rather than inventing fake percentages or fake company names. Furthermore, the UI requires Human-in-the-Loop approval (Accept, Edit, or Reject).

Q9. How does the AI Job Discovery Engine build the Candidate Profile Context?

In candidate_intelligence.py, build_candidate_profile_snapshot() aggregates the user's Master Profile AND all saved active resumes. It deduplicates skills, projects, and experiences, tracks provenance ('master_profile' vs 'resume:<id>:<title>'), prefers Master Profile data on conflict, and flags contradictions (like mismatched graduation years or locations across resumes) in resume_conflicts_json.

Q10. How does the Job Discovery Engine prevent duplicate jobs when the same role is on multiple sites?

In job_sources.py, compute_canonical_job_fingerprint() normalizes the company name (stripping suffixes like Inc/LLC/Labs) and role title, then computes a SHA-256 hash stored in JobListing.canonical_fingerprint (unique index). When the same opening is found on another source, we attach a new JobSourceEvidence record to the existing JobListing instead of creating a duplicate card.

Q11. How does the system select the 'Best Resume for This Job'?

In job_matcher_ranker.py, select_best_resume_for_job() compares every saved resume's extracted skills and strong-evidence count against the job's required and preferred skills, plus headline similarity, returning the highest-scoring resume ID, title, explanation, and a side-by-side comparison array.

Q12. What prevents a designer who knows basic Python from receiving hundreds of backend Python jobs?

Two mechanisms in candidate_intelligence.py and job_matcher_ranker.py: (1) Dominant-domain detection checks the candidate's headline and STRONG evidence skills so a design-dominant profile isn't flooded with backend roles from a secondary LISTED skill, and (2) UserJobPreferences provides an explicit 'override_inferred_roles' toggle and 'excluded_categories' filter.

Q13. How does the Dual Gmail SMTP Verification system work?

When a user registers (/api/auth/register), we generate both a 6-digit numeric OTP (secrets.randbelow(900000) + 100000, valid for 15 minutes) and a 32-byte URL-safe token (secrets.token_urlsafe(32), valid for 24 hours). Both are sent in a single HTML/Plain-text email via FastAPI BackgroundTasks so registration responds immediately without blocking.

Q14. What is 'Ghost Account Recovery' in your registration flow?

If a user registers with an email, closes the tab before verifying, and tries to register again later, most apps throw 'Email already exists' and lock the user out. In backend/app/api/auth.py (lines 195-205), if existing_user.is_verified is False, we update their password, generate a fresh OTP and token, and re-send the verification email seamlessly.

Q15. How do you protect the OTP resend endpoint from spam/abuse?

We store last_verification_sent_at on the User model and enforce a strict 60-second cooldown in /api/auth/resend-verification, returning HTTP 429 Too Many Requests with the exact remaining seconds if called early.

Q16. Why does email_service.py try Port 587 first and fallback to Port 465?

Many college or corporate firewalls block outbound STARTTLS on port 587 or SSL on port 465. create_smtp_connection() attempts port 587 with STARTTLS first, and if a socket/firewall error occurs, automatically retries over implicit SSL (SMTPS) on port 465. If no SMTP credentials are configured, it prints the OTP and link clearly in the server terminal.

Q17. How does JD Import via URL or File work in the Job Match section?

In backend/app/api/job.py, POST /api/job/extract-file uses PyMuPDF/python-docx to parse uploaded .pdf, .docx, or .txt JDs. POST /api/job/extract-url validates the URL against SSRF, fetches the page, extracts Schema.org JobPosting JSON-LD (used by Greenhouse, Lever, LinkedIn, Indeed) or HTML/PDF body text, infers the Role Title and Company, and auto-populates the frontend inputs.

Q18. How is authentication and user isolation enforced across all endpoints?

Every protected route injects current_user: User = Depends(get_current_active_user). The JWT token is signed with HMAC-SHA256 (HS256), passwords are salted and hashed with Bcrypt, and every database query filters by model.user_id == current_user.id so User B can never access User A's resumes, analyses, saved jobs, or preferences.

Q19. Why did you use SQLAlchemy 2.0 and how many tables are in the database?

We use SQLAlchemy 2.0 ORM with 22 relational tables (13 core tables: users, profiles, educations, experiences, projects, user_skills, skills, skill_aliases, resumes, job_descriptions, job_skills, analysis_history, skill_gaps, recommendations; plus 9 job discovery tables: job_sources, job_listings, job_source_evidences, candidate_profile_snapshots, user_job_preferences, job_search_runs, job_recommendations, saved_jobs, job_applications). It works zero-config with SQLite locally and switches to PostgreSQL via DATABASE_URL.

Q20. How does FastAPI's Dependency Injection (Depends) work in your backend?

FastAPI's Depends() manages per-request resources and security guards. get_db() yields a SQLAlchemy Session and guarantees db.close() in a finally block after the request finishes. get_current_active_user() decodes the Bearer JWT token, queries the User, and verifies user.is_active before the route handler runs.

Q21. Why does the Resume Builder export single-column PDFs using ReportLab instead of browser window.print()?

Browser window.print() produces inconsistent margins, headers/footers, and CSS artifacts across Chrome/Edge/Firefox that confuse enterprise ATS parsers. Our server-side ReportLab engine (backend/app/services/pdf_service.py) generates a deterministic, machine-readable single-column PDF stream with embedded clickable hyperlinks.

Q22. What happens if the college Wi-Fi has no internet or blocks external APIs during the offline review?

SkillBit is engineered to be 100% offline-resilient: (1) Database runs on local SQLite (app.db), (2) NLP parsing, skill extraction, and TF-IDF matching run 100% locally in Python, (3) LLM calls automatically fall back to the deterministic SkillBit Local CareerTech Engine, (4) Job Discovery includes a pre-seeded Verified Career Index Cache, and (5) SMTP verification prints the 6-digit OTP directly in the backend terminal if SMTP is unreachable.